

# SECURITY REVIEW

**FOR MYSTIC FINANCE**



mystic<sup>★</sup>

## INSIDE THIS REPORT

# CONTENTS

|    |                                                                                                                               |    |
|----|-------------------------------------------------------------------------------------------------------------------------------|----|
| 01 | AUDIT SUMMARY                                                                                                                 | 03 |
| 02 | SCOPE                                                                                                                         | 04 |
| 03 | AUDITORS                                                                                                                      | 05 |
| 04 | METHODOLOGY                                                                                                                   | 06 |
| 05 | RISK CLASSIFICATION                                                                                                           | 07 |
| 06 | FINDINGS SUMMARY                                                                                                              | 08 |
| 07 | DETAILED FINDINGS                                                                                                             | 09 |
|    | H-01 Incorrect Accounting of currentWithheldETH in Instant Withdrawals                                                        | 09 |
|    | H-02 User Funds Burned With No ETH Returned Due to Missing withdrawalRequests[msg.sender] Assignment After Unstake            | 10 |
|    | M-01 Incorrect increasing of end timestamp causes OverExtended delay in syncRewards()                                         | 11 |
|    | M-02 _depositEther Does Not Increment Validator Index, Causing All Deposits to Funnel Into First Validator and Fail Once Full | 12 |
|    | I-01 submitForValidator() Blocked by Strategic Capacity Manipulation                                                          | 13 |
|    | I-02 Missing Length Consistency Check Between amounts and tokens Arrays in claimAll()                                         | 14 |
|    | I-03 AllRewardsClaimed is not emitted when all rewards are claimed                                                            | 15 |
|    | I-04 <code>withdrawGov</code> will always return zero                                                                         | 16 |
| 08 | DISCLAIMER                                                                                                                    | 17 |
| 09 | ABOUT KANN AUDITS                                                                                                             | 18 |

## 01 - OVERVIEW

# AUDIT SUMMARY

Mystic Finance engaged Kann Audits to perform a security review of the Mystic Finance smart contracts. A team of 3 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

## TOTAL FINDINGS

# 8

ISSUES

|               |    |
|---------------|----|
| CRITICAL      | -  |
| HIGH          | 02 |
| MEDIUM        | 02 |
| LOW           | -  |
| INFORMATIONAL | 04 |

## ENGAGEMENT DETAILS

|               |                |
|---------------|----------------|
| PREPARED FOR  | Mystic Finance |
| PREPARED BY   | Kann Audits    |
| DATE REPORTED | 8 June 2025    |
| METHODS       | Manual review  |

## 02 - WHAT WE REVIEWED

# SCOPE

|                |                                                                                                                                                     |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| REPOSITORY     | <a href="https://github.com/mystic-finance/Liquid-Staking/tree/staked-plume">https://github.com/mystic-finance/Liquid-Staking/tree/staked-plume</a> |
| AUDITED COMMIT | a7a86bdc3d65c5c2512fe6a835c10b4bc2d4315d                                                                                                            |

| #  | FILE                   |
|----|------------------------|
| 01 | /src/stPlumeMinter.sol |
| 02 | /src/frxETH.sol        |

03 - THE TEAM ON THIS ENGAGEMENT

# AUDITORS

KA

**Kann**

LEAD SECURITY RESEARCHER

ST

**Strapontin**

ASSOCIATE SECURITY RESEARCHER

TR

**TradMod**

ASSOCIATE SECURITY RESEARCHER

# METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

## STEP BY STEP

### OUR SECURITY PROCESS

01

#### Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

#### Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

#### AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

#### Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

#### Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

#### Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

## COVERAGE

### THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

# RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

|                 |                                                                                                                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ● CRITICAL      | Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.                                   |
| ● HIGH          | Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.                                                      |
| ● MEDIUM        | Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.            |
| ● LOW           | Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.                                                                      |
| ● INFORMATIONAL | Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability. |

06 - ALL ISSUES

# FINDINGS SUMMARY

|                                         |        |          |       |        |
|-----------------------------------------|--------|----------|-------|--------|
| ● CRITICAL                              | ● HIGH | ● MEDIUM | ● LOW | ● INFO |
| -                                       | 02     | 02       | -     | 04     |
| TOTAL 08    FIXED 04    ACKNOWLEDGED 04 |        |          |       |        |

| ID   | TITLE                                                                                                                    | SEVERITY        | STATUS         |
|------|--------------------------------------------------------------------------------------------------------------------------|-----------------|----------------|
| H-01 | Incorrect Accounting of currentWithheldETH in Instant Withdrawals                                                        | ● HIGH          | ● FIXED        |
| H-02 | User Funds Burned With No ETH Returned Due to Missing withdrawalRequests[msg.sender] Assignment After Unstake            | ● HIGH          | ● FIXED        |
| M-01 | Incorrect increasing of end timestamp causes OverExtended delay in syncRewards()                                         | ● MEDIUM        | ● FIXED        |
| M-02 | _depositEther Does Not Increment Validator Index, Causing All Deposits to Funnel Into First Validator and Fail Once Full | ● MEDIUM        | ● FIXED        |
| I-01 | submitForValidator() Blocked by Strategic Capacity Manipulation                                                          | ● INFORMATIONAL | ● ACKNOWLEDGED |
| I-02 | Missing Length Consistency Check Between amounts and tokens Arrays in claimAll()                                         | ● INFORMATIONAL | ● ACKNOWLEDGED |
| I-03 | AllRewardsClaimed is not emitted when all rewards are claimed                                                            | ● INFORMATIONAL | ● ACKNOWLEDGED |
| I-04 | <code>withdrawGov</code> will always return zero                                                                         | ● INFORMATIONAL | ● ACKNOWLEDGED |

# DETAILED FINDINGS

H-01

## Incorrect Accounting of currentWithheldETH in Instant Withdrawals

SEVERITY

● HIGH

STATUS

● FIXED

### ■ DESCRIPTION

In the `withdraw(address recipient)` function, when the withdrawal amount is less than or equal to `currentWithheldETH`, the function enters the instant withdrawal path. In this case, the contract does not call `plumeStaking.withdraw()`, and the funds are expected to be fully covered by the ETH already held in `currentWithheldETH`. However, the code still sets `withdrawn = amount` and then unconditionally adds this value back to `currentWithheldETH`. Immediately after, it subtracts `amount` from `currentWithheldETH`, effectively leaving the value of `currentWithheldETH` unchanged.

```
} else {  
    fee = amount * INSTANT_REDEMPTION_FEE / RATIO_PRECISION;  
    withdrawn = amount;  
}  
uint256 cachedWithheldETH = currentWithheldETH;  
currentWithheldETH += withdrawn; //adding the withdraw amount  
withHoldEth += fee;  
currentWithheldETH -= amount; //deducting the amount
```

This logic is flawed because the ETH used to fulfill the withdrawal is not newly received; it is already present in `currentWithheldETH`, and should simply be subtracted, not first added and then subtracted. The current logic makes it appear as if no ETH was withdrawn, which results in incorrect accounting of the withheld pool. If multiple such withdrawals occur, the system will consistently over-report the available withheld ETH, potentially leading to over-withdrawals or failed withdrawals in the future when actual funds are not available.

### ■ REMEDIATION

● FIXED

H-02

## User Funds Burned With No ETH Returned Due to Missing withdrawalRequests[msg.sender] Assignment After Unstake

SEVERITY

● HIGH

STATUS

● FIXED

### ■ DESCRIPTION

The unstaking process is executed in two steps:

The user initiates unstake().

If currentWithheldETH is insufficient, the contract attempts to unstake from the Plume validator.

However, during the fallback logic that handles validator unstaking, the contract returns amountUnstaked without saving it in the withdrawalRequests[msg.sender] mapping. This causes a critical failure in the withdrawal flow.

Users who unstake in this path will have their frxETH tokens burned but will receive no ETH in return because

No withdrawalRequest is saved.

The withdraw() function depends on withdrawalRequests[msg.sender] being populated.

This effectively results in loss of user funds with no means of recovery through the intended withdraw flow.

### ■ REMEDIATION

● FIXED

M-01

## Incorrect increasing of end timestamp causes OverExtended delay in syncRewards()

SEVERITY

● MEDIUM

STATUS

● FIXED

### ■ DESCRIPTION

The syncRewards() function contains faulty logic that causes the rewardsCycleEnd to be pushed one full cycle further than necessary. When the function is called late into the current cycle - for example at timestamp 180 with a rewardsCycleLength of 100 - it calculates the next rewardsCycleEnd as 200, but then adds an extra 100, setting it to 300. This happens because of the condition:

```
if (end - timestamp < rewardsCycleLength) {  
    end += rewardsCycleLength;  
}
```

At timestamp 180, the current cycle (ending at 100) is already complete, and only 20 seconds remain to complete the next one. However, instead of moving to 200 as expected, the function pushes the end to 300, delaying the next sync by a full cycle. This causes up to nearly one full cycle of rewards to be skipped and forces users to wait longer than necessary to receive rewards.

### ■ REMEDIATION

● FIXED

M-02

## **\_depositEther Does Not Increment Validator Index, Causing All Deposits to Funnel Into First Validator and Fail Once Full**

SEVERITY

● MEDIUM

STATUS

● FIXED

### ■ DESCRIPTION

In the stPlumeMinter contract, the `_depositEther()` function is responsible for distributing incoming ETH deposits across available validators. This function loops through the validator set to find suitable validators with enough capacity to stake the deposited ETH.

However, there is a critical issue: the index variable, initialized to 0, is never incremented inside the while loop. As a result, the loop evaluates the same validator (`validators[0]`) in every iteration, preventing the function from ever progressing to the next validator in the list.

When the first validator reaches its staking capacity or becomes inactive, the `getNextValidator()` function returns a capacity of 0. Since `depositSize < minStakeAmount` in such cases, the full remaining-Amount is stored in the `currentWithheldETH` variable. This means the user's deposit is held back and never actually staked, despite the user receiving yield-bearing tokens.

### ■ REMEDIATION

● FIXED

I-01

## submitForValidator() Blocked by Strategic Capacity Manipulation

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

### ■ DESCRIPTION

Intentionally people could leave capacity almost to max - (min stake value + 1) so nobody could call submitForValidator() for this exact validator. Which will result in logic going always on increasing the currentWithheldETH

### ■ REMEDIATION

● ACKNOWLEDGED

I-02

## Missing Length Consistency Check Between amounts and tokens Arrays in claimAll()

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

### ■ DESCRIPTION

In the `claimAll()` function, the `amounts` and `tokens` arrays are expected to correspond to each other by index. However, the code does not validate that both arrays are of the same length before using them in a for loop. If `plumeStaking.claimAll()` and `plumeStaking.getRewardTokens()` return arrays of different lengths, this can lead to out-of-bounds access or incorrect logic execution.

To prevent potential errors or misaligned reward handling, a check should be added:

```
require(amounts.length === tokens.length, "Mismatch between amounts and tokens");
```

### ■ REMEDIATION

● ACKNOWLEDGED

I-03

## AllRewardsClaimed is not emitted when all rewards are claimed

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

### ■ DESCRIPTION

In `stPlumeMinter` there is a function called `claimAll` that is used to claim rewards accumulated from all rewards tokens, but here is the case that the event `AllRewardsCLaimed` is not emitted after all rewards have been claimed

### ■ REMEDIATION

● ACKNOWLEDGED

I-04

**withdrawGov will always return zero**

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

## ■ DESCRIPTION

This function is supposed to return how much owner or timelock(i.e Gov) withdrew from `plumeStaking` but the return value `amountRestaked` is not updated in the function and as a result it always returns zero.

```
function withdrawGov(uint256 amount) external nonReentrant onlyByOwnGov returns (
    uint256 amountRestaked) {
    _rebalance();
    uint256 balanceBefore = address(this).balance;
    plumeStaking.withdraw();
    uint256 balanceAfter = address(this).balance;
    currentWithheldETH += balanceAfter - balanceBefore;
}
```

## ■ REMEDIATION

● ACKNOWLEDGED

# DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

# ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



## EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



## AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



## FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



## INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



**KANN AUDITS**  
WEB3 SECURITY AUDITS

### GET IN TOUCH

|           |                                                                    |
|-----------|--------------------------------------------------------------------|
| WEBSITE   | <a href="https://kannaudits.com">kannaudits.com</a>                |
| PORTFOLIO | <a href="https://github.com/KannAudits">github.com/KannAudits</a>  |
| EMAIL     | <a href="mailto:contact@kannaudits.com">contact@kannaudits.com</a> |
| X         | <a href="https://twitter.com/KannAudits">@KannAudits</a>           |
| TELEGRAM  | <a href="https://t.me/KannAudits">@KannAudits</a>                  |